

MODELAGEM E AVALIAÇÃO DE ARQUITETURA DE SOFTWARE PARA SISTEMAS WEB ESCALÁVEIS

SOFTWARE ARCHITECTURE MODELING AND EVALUATION FOR SCALABLE WEB SYSTEMS

Kevin Payão Reisauskas^{1, i}
Deivison Shindi Takatu^{2, ii}
Glauco Todesco^{3, iii}

RESUMO

O presente trabalho aborda o desafio da degradação de desempenho em aplicações web submetidas a alta carga computacional, um problema recorrente em arquiteturas legadas. O objetivo central é projetar e avaliar a escalabilidade de uma API REST (Laravel) utilizando uma arquitetura *cloud-native serverless* no Amazon ECS com AWS Fargate, comparando-a diretamente com uma *baseline* local orquestrada via Docker Compose. A pesquisa adotou uma metodologia quantitativa e experimental, empregando o script K6 para simular o tráfego de 100 Usuários Virtuais (VUs) simultâneos sob um *Service Level Agreement* (SLA) rigoroso de 500 milissegundos. Os resultados dos testes indicaram o rápido esgotamento dos recursos da *baseline* local, que atingiu uma latência máxima de 22.234 ms e obteve 0% de conformidade com o SLA exigido. Conclui-se que o ambiente monolítico de capacidade fixa é incapaz de suportar picos de utilização, o que justifica empiricamente a necessidade da migração, atualmente em implantação, para a arquitetura elástica e distribuída da Amazon Web Services (AWS), visando garantir alta disponibilidade, resiliência e a otimização de recursos provisionados apenas sob demanda.

Palavras-chave: Computação em Nuvem. *Serverless*. Escalabilidade. Arquitetura de Software. AWS.

ABSTRACT

This paper addresses the challenge of performance degradation in web applications subjected to high computational loads, a recurring problem in legacy architectures. The main objective is to design and evaluate the scalability of a REST API (Laravel) using a cloud-native serverless architecture on Amazon ECS with AWS Fargate, comparing it directly with a local baseline orchestrated via Docker Compose. The research adopted a quantitative and experimental methodology, using the K6 script to simulate the traffic of 100 simultaneous Virtual Users (VUs) under a strict Service Level Agreement (SLA) of 500 milliseconds. The test results indicated the rapid exhaustion of the local baseline's resources, which reached a maximum latency of

¹ Graduando em Análise e Desenvolvimento de Sistemas na Faculdade SENAI Gaspar Ricardo Júnior. E-mail: kevin.reisauskas@gmail.com

² Mestre em Ciência da Computação pela UFSCar Sorocaba e Docente de Análise e Desenvolvimento de Sistemas da Faculdade SENAI Gaspar Ricardo Júnior. E-mail: deivison.takatu@sp.senai.br

³ Doutor em Engenharia Elétrica pela Universidade de São Paulo e Docente de Análise e Desenvolvimento de Sistemas da Faculdade SENAI Gaspar Ricardo Júnior. E-mail: glauco.todesco@sp.senai.br

22,234 ms and obtained 0% compliance with the required SLA. It is concluded that the fixed-capacity monolithic environment is unable to support peak utilization, empirically justifying the need for the ongoing migration to the elastic and distributed architecture of Amazon Web Services (AWS), aiming to ensure high availability, resilience, and the optimization of resources provisioned only on demand.

Keywords: Cloud Computing. Serverless. Scalability. Software Architecture. AWS.

1 INTRODUÇÃO

O dimensionamento de APIs web para suportar variações repentinas de tráfego representa um desafio crítico na engenharia de software contemporânea. Sistemas monolíticos e hospedados em infraestruturas tradicionais enfrentam frequentemente o dilema entre o desperdício financeiro gerado pela manutenção de recursos ociosos ou a degradação crítica de desempenho sob cargas de tráfego intenso.

Para superar esses obstáculos, a transição para arquiteturas orientadas a microsserviços e nativas em nuvem (*cloud-native*) tornou-se uma estratégia indispensável, permitindo o escalonamento dinâmico de instâncias de processamento em tempo real. Nesse contexto, o presente estudo investiga a transição de um ecossistema de infraestrutura estática para um modelo dinâmico na nuvem.

1.1 Problema de pesquisa

O problema central desta pesquisa reside no elevado custo operacional e na alta latência de resposta inerentes a infraestruturas computacionais rígidas, que são incapazes de se adaptar instantaneamente a picos imprevisíveis de uso, resultando na degradação sistêmica e no esgotamento dos recursos locais ou verticalizados.

1.2 Objetivo

O objetivo principal deste projeto é projetar e avaliar a escalabilidade de uma API REST desenvolvida em Laravel operando em uma arquitetura *cloud-native serverless* (utilizando AWS ECS Fargate), realizando um estudo comparativo quantitativo contra uma *baseline* local orquestrada via Docker Compose.

1.3 Justificativa

A justificativa para a execução deste trabalho pauta-se na premente necessidade de otimização econômica no provisionamento de recursos tecnológicos, associada à garantia rigorosa do SLA (*Service Level Agreement*) de tempo de resposta. A adoção de modelos que consomem recursos sob demanda garante a sustentabilidade financeira e técnica das aplicações web modernas.

2 REVISÃO DE LITERATURA

A Computação em Nuvem transformou o gerenciamento de infraestrutura ao permitir a alocação de recursos sob demanda. De acordo com as diretrizes arquiteturais delineadas por Pahl, Jamshidi e Zimmermann (2018), aplicações *cloud-native* fundam-se na combinação de orientação a serviços, rápida implantação e virtualização, frequentemente utilizando microsserviços como formato

de composição para superar o forte acoplamento dos monólitos. No âmbito de virtualização e abstração, Eismann et al. (2020) destacam o avanço do modelo *Serverless* e da Computação em Nuvem, que ocultam a gestão e a manutenção de servidores dos desenvolvedores, permitindo um provisionamento elástico sob demanda (cobrado pelo uso real), o que se mostra uma abordagem altamente eficiente para cargas de trabalho irregulares ou passíveis de grandes picos.

Para que essa distribuição descentralizada e elástica ocorra com baixo impacto sobre a rede e processamento, as unidades de execução devem ser leves e isoladas. Conforme elucidado por Bernstein (2014), o uso de contêineres como o Docker permite encapsular a aplicação e suas dependências de maneira portátil e com isolamento de processos, substituindo a pesada camada das máquinas virtuais tradicionais e padronizando os ciclos de implantação.

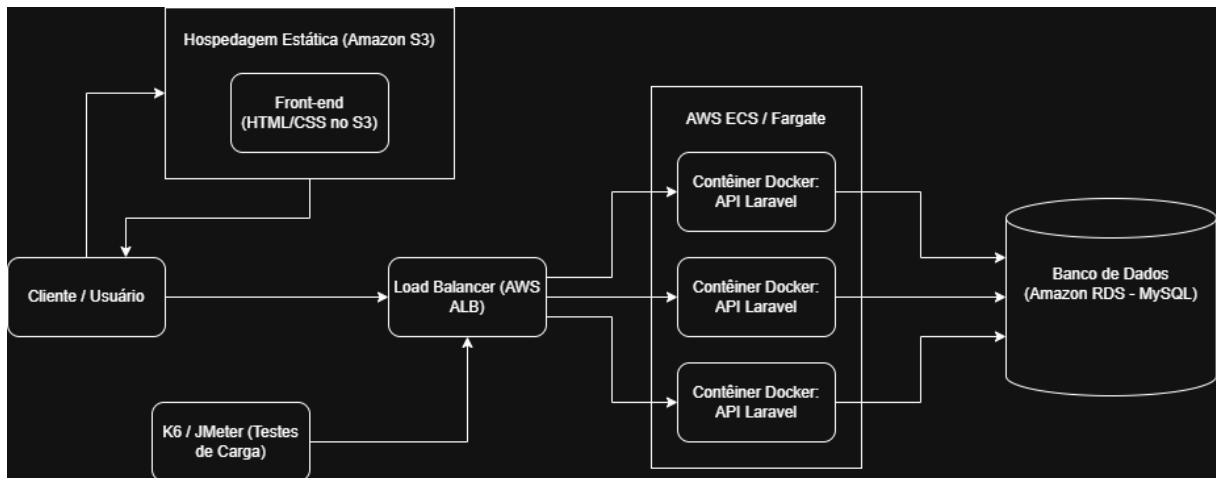
Contudo, a elasticidade dinâmica de microsserviços, porém, introduz latência de rede e variabilidade de carga que tornam o dimensionamento teórico insuficiente. Heinrich et al. (2017) argumentam que a engenharia de desempenho nesse contexto depende de monitoramento contínuo e testes de carga iterativos para calibrar a capacidade real de cada serviço. Blinowski, Ojdowska e Przybyłek (2022) corroboram empiricamente: testes de carga controlados são necessários para identificar o ponto em que a capacidade computacional (seja local ou em nuvem) é excedida e a latência de resposta é degradada.

3 METODOLOGIA

A presente pesquisa caracteriza-se como um estudo de natureza quantitativa e experimental, apoiado na simulação de tráfego, coleta métrica de sistemas e análise de desempenho. Inicialmente, empregou-se a ferramenta Docker Compose para orquestrar e estabelecer a *baseline* em um ambiente local controlado, isolando a API e o banco de dados em contêineres próprios a fim de registrar o comportamento do sistema sem distribuição elástica.

A proposta principal, atualmente em fase de implantação, consiste na migração para uma arquitetura *cloud-native* distribuída nos serviços da Amazon Web Services (AWS). Nessa arquitetura, a entrega de arquivos estáticos do front-end é delegada ao Amazon S3. O tráfego de requisições destinado à API é centralizado e roteado por um *Application Load Balancer* (ALB). Para o processamento da lógica de negócios, a aplicação utilizará a computação *serverless* através do AWS ECS Fargate, permitindo que os contêineres do back-end escalem horizontalmente em resposta ao aumento de carga de CPU sem a necessidade de provisionar ou gerenciar servidores virtuais subjacentes. Por fim, os dados persistentes serão mantidos e gerenciados pelo Amazon RDS.

Figura 1 – Arquitetura cloud-native em implantação



Fonte: Elaborado pelo autor (2026).

Para o teste de estresse da *baseline* local, foi desenvolvida uma suíte de testes utilizando a ferramenta de testes de carga K6. A configuração do script foi estruturada para avaliar o sistema por meio de estágios predefinidos: inicia-se com um período de *Ramp-up* escalando de 0 a 50 Usuários Virtuais (VUs) ao longo de 10 segundos. Na sequência, há a etapa de Estresse máximo, onde a carga salta para 100 VUs simultâneos, sendo sustentada por 30 segundos. Posteriormente, a injeção é estabilizada na marca de 100 VUs por mais 10 segundos, finalizando com o desligamento gradual (*Ramp-down*) até 0 VUs nos últimos 10 segundos. O SLA (*Service Level Agreement*) de aceitação do sistema foi estipulado como sendo um tempo de resposta máximo de 500ms para 95% das requisições.

Tabela 1 — Parâmetros do Script de Teste de Carga (K6)

Estágio	Duração	Carga Alvo	Objetivo
Ramp-up	10s	0 a 50 VUs	Aquecimento gradual da API
Pico (Stress)	30s	100 VUs	Injeção máxima de carga
Sustentação	10s	100 VUs	Avaliação de resiliência
Ramp-down	10s	100 a 0 VUs	Desligamento gradual

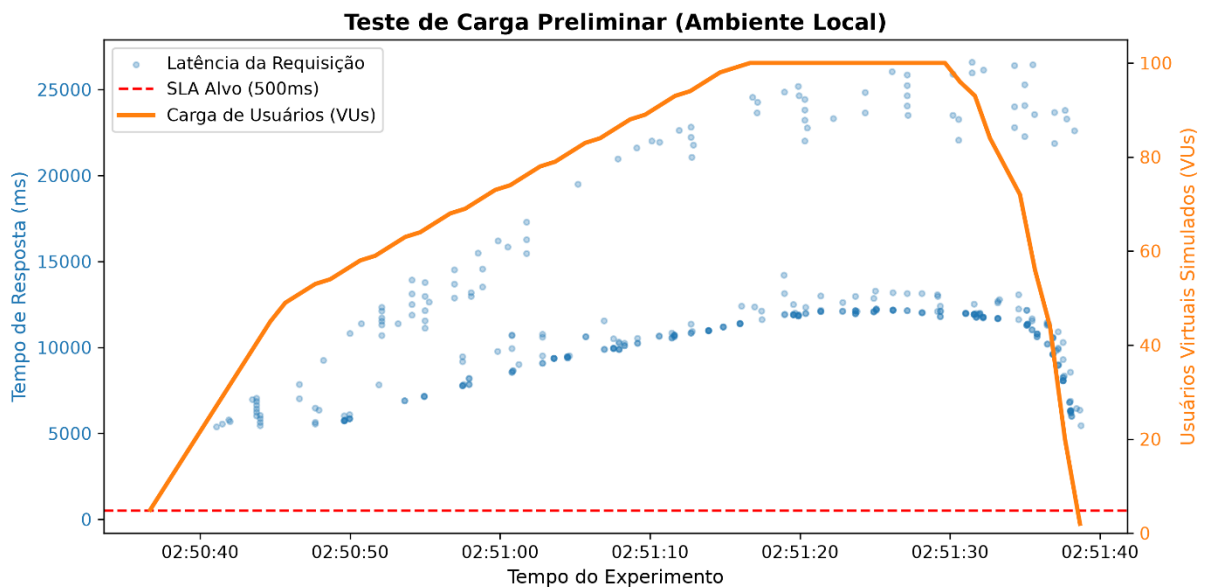
Fonte: Elaborado pelo autor (2026).

4 RESULTADOS PARCIAIS

Durante o teste de carga no ambiente local, observou-se uma rápida deterioração do desempenho à medida que a quantidade de requisições simultâneas aumentou. Durante a injeção da carga parametrizada nos estágios do K6, a API conseguiu processar o tráfego a ponto de retornar 100% dos status de resposta como HTTP 200 (sucesso lógico), não apresentando erros ou indisponibilidades de conexão

(downtime absoluto).
Todavia, ao alcançar o pico programado de 100 VUs simultâneos, o desempenho do sistema sofreu uma degradação severa, refletindo o esgotamento dos recursos locais. A latência máxima da API atingiu alarmantes 22.234 ms (pouco mais de 22 segundos), rompendo o limite planejado em aproximadamente 27 vezes.

Figura 2 – Performance da Baseline Local sob Carga de 100 VUs



Fonte: Elaborado pelo autor (2026).

Devido à magnitude desse gargalo de tempo de resposta, os testes resultaram em 0% de conformidade com o SLA exigido de 500ms, apontando falha crítica no critério de aceitação de performance.

Tabela 2 — Síntese das Métricas da Baseline Local

Métrica Analisada	Valor Obtido	Status
Latência Máxima Observada	22.234 ms	Reprovado
Taxa de Sucesso (HTTP 200)	100%	Aprovado
Conformidade com SLA (< 500ms)	0%	Falha Crítica

Fonte: Elaborado pelo autor (2026).

Os resultados obtidos evidenciam que um servidor único com capacidade de processamento fixa não consegue absorver picos de tráfego concorrente. Embora a aplicação não tenha retornado erros HTTP (mantendo 100% de respostas com status 200), o tempo de espera tornou o serviço impraticável para o usuário final.
A alta latência mostra empiricamente a inability de sustentar picos imprevisíveis ou cargas massivas sem a presença de mecanismos adequados de balanceamento e distribuição de carga. Estes resultados locais fundamentam o problema de pesquisa delineado, além de validar a necessidade

estrutural de migração para o ecossistema distribuído, elástico e *serverless* do AWS Fargate, justificando a adoção da arquitetura proposta.

5 CONCLUSÕES PRELIMINARES

O desenvolvimento até a presente etapa permite concluir que as metodologias de containerização foram bem-sucedidas no isolamento e na padronização da aplicação. Entretanto, o ambiente em escopo local e monolítico expôs falhas críticas de performance sob intensa concorrência, não comportando a carga simulada e extrapolando o limite de aceitação do tempo de resposta. Como continuidade desta Iniciação Científica, os mesmos cenários de estresse serão replicados na infraestrutura em nuvem (AWS ECS Fargate), permitindo confrontar diretamente o comportamento de uma aplicação elástica contra a *baseline* local obtida até o momento.

REFERÊNCIAS

BERNSTEIN, David. Containers and cloud: From LXC to Docker to Kubernetes. **IEEE Cloud Computing**, v. 1, n. 3, p. 81-84, 2014. Disponível em: <https://sweet.ua.pt/andre.zuquete/Aulas/AES/20-21/extras/Bernstein14.pdf>. Acesso em: 22 abr. 2026.

BLINOWSKI, Grzegorz; OJDOWSKA, Anna; PRZYBYŁEK, Adam. Monolithic vs. microservice architecture: A performance and scalability evaluation. **IEEE Access**, v. 10, p. 20357-20374, 2022. Disponível em: <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9717259>. Acesso em: 08 maio 2026.

HEINRICH, Robert et al. Performance engineering for microservices: research challenges and directions. In: **Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion**. 2017. p. 223-226. Disponível em: <https://dl.acm.org/doi/pdf/10.1145/3053600.3053653>. Acesso em: 22 abr. 2026.

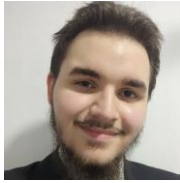
PAHL, Claus; JAMSHIDI, Pooyan; ZIMMERMANN, Olaf. Architectural principles for cloud software. **ACM Transactions on Internet Technology (TOIT)**, v. 18, n. 2, p. 1-23, 2018. Disponível em: <https://dl.acm.org/doi/pdf/10.1145/3104028>. Acesso em: 22 abr. 2026.

AGRADECIMENTOS

Agradeço primeiramente aos meus orientadores, Prof. Deivison Shindi Takatu e Prof. Glauco Todesco, por todo o suporte, paciência e direcionamento técnico fundamentais para a condução desta iniciação científica, e à Faculdade SENAI Gaspar Ricardo Júnior pela infraestrutura e fomento à pesquisa acadêmica, possibilitando a realização deste estudo.

SOBRE OS AUTORES

ⁱ Kevin Payão Reisauskas



Graduando do curso de Análise e Desenvolvimento de Sistemas pela Faculdade SENAI Gaspar Ricardo Júnior. Possui experiência prática em desenvolvimento de software voltado para a web, com foco na construção de APIs RESTful. Atualmente, dedica-se ao estudo de arquiteturas nativas em nuvem, containerização com Docker e metodologias para escalabilidade de sistemas distribuídos na Amazon Web Services (AWS).

<https://orcid.org/0009-0009-2881-0893>

ii **Deivison Shindi Takatu**



Possui graduação em Análise e Desenvolvimento de Sistemas (2016) e Licenciatura em Matemática (2023), com especialização em Informática Aplicada à Educação pelo IFSP (2018). É Mestre em Ciência da Computação pela UFSCar – Campus Sorocaba (2021). Atualmente é professor universitário no SENAI Gaspar Ricardo Júnior, com experiência em Ciência da Computação e Tecnologias Educacionais.

<https://orcid.org/0000-0002-8210-1002>

iii **Glauco Todesco**



Possui graduação em Tecnologia em Processamento de Dados pela FATEC Sorocaba (1995), mestrado em Ciência da Computação pela UFSCar (2000) e doutorado em Engenharia Elétrica pela USP (2006). Atua como professor de graduação na área de Computação, incluindo o SENAI Gaspar Ricardo Júnior, com experiência em pesquisa, desenvolvimento de software e educação.

<https://orcid.org/0000-0003-0603-0641>